Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 6

Disciplina: Tehnici de programare

Tema: ” Mini-arsenal de forensics: decodare, XOR, hash-uri, metadate, file carving, steganografie LSB”

A efectuat : st. gr.SI – 261
Talchiv Corina

A verificat : asistent universitar
Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

Două unelte de forensics, scrise de voi în Python, care intră în arsenal. Lucrarea A este un decoder universal: dat un șir, recunoaște stratul (base64, hex, URL), îl desface, iar dacă e un XOR cu cheie scurtă sau un hash de parolă, îl sparge prin forță brută sau dicționar. Lucrarea B este forensics pe fișiere: citești semnătura reală, scoți textul și metadatele (inclusiv GPS) dintr-o poză, extragi un fișier ascuns, apoi ascunzi și dezvălui un mesaj prin steganografie LSB. Amândouă se predau cum cere regula de fier: cod și output, niciodată o captură de ecran.

LUCRAREA A (nucleu). Decoderul universal: arsenal/forensics/decode.py

Obiectiv: un program care primește un șir sau un fișier, recunoaște stratul și îl desface; iar pentru un XOR cu cheie de un octet sau un hash de parolă, îl sparge singur. Pornești de la scheletul de mai jos; funcțiile marcate cu TODO sunt ale tale.

LUCRAREA B (extindere). Forensics pe fișiere și un mesaj ascuns

Obiectiv: citești ce spun fișierele despre ele (semnătură, text, metadate), extragi un fișier ascuns, apoi ascunzi și dezvălui un mesaj prin steganografie LSB. Se lucrează pe fișierele din probe/.

Cum predai (obligatoriu, prin git):

Urci TOT codul în depozitul tău git tehnici-prog-<nume> (cadrul didactic este colaborator) și dai commit + push la finalul lucrării. Asistentul îl descarcă de acolo (git pull) și îl rulează. Nu se trimite nimic pe e-mail și nicio captură de ecran.

```
git add .
git commit -m "Lab 6: decoder universal + forensics si stego"
git push
```

Nucleul (Lucrarea A) se face pe viu, în laborator. Extinderea (Lucrarea B) se poate continua acasă: fișierele din probe/ sunt generate de tine, deci nu depinzi de rețeaua laboratorului. Se notează codul care rulează și output-ul, nu capturi de ecran.

2. Desfășurarea lucrării

Partea A (decode.py) se ocupă de decodare. Am început cu conversia dintre text și octeți, folosind .encode(), .hex() și bytes.fromhex(). Apoi funcția ghicim_stratul recunoaște encodingul după formă: % înseamnă url, doar cifre hex cu lungime pară înseamnă hex, iar alfabetul base64 cu lungimea multiplu de 4 înseamnă base64. Am verificat hex înaintea base64, fiindcă un șir hex poate fi și base64 valid. Funcția desfacem aplică decodarea potrivită, iar citibil verifică dacă toți octeții sunt ASCII imprimabil (32–126), ca să știu când am ajuns la textul clar. Pe aceeași idee, sparge_xor încearcă toate cele 256 de chei și păstrează textul imprimabil, iar sparge_hash alege algoritmul după lungimea hash-ului (32 → md5, 40 → sha1, altfel sha256)

și compară hash-ul fiecărui cuvânt din wordlist.txt cu ținta. Programul se rulează cu argparse (un șir, --fisier sau --sparge-hash) și salvează rezultatul în fisa.json.
Partea B pornește de la ideea că un fișier poate ascunde mai mult decât se vede. În forensics.py, tip_real stabilește tipul fișierului din primii octeți (%PDF, \x89PNG, \xff\xd8, PK\x03\x04), nu din extensie, iar expresia regulată [-~]{4,} scoate textul imprimabil dintr-un binar. Din foto.jpg am citit metadatele EXIF și am convertit coordonatele GPS în grade zecimale cu formula grade + minute/60 + secunde/3600, cu minus pentru S și W. Pentru file carving am căutat semnătura PK\x03\x04 în ascuns.png și am salvat arhiva ZIP de la acel octet până la final. În stego.py am ascuns un mesaj în pixeli: l-am transformat în biți, cu un octet NUL la final ca terminator, și am scris fiecare bit în ultimul bit al unui canal de culoare (R, G, B). La citire adun LSB-urile, le grupez câte 8 și mă opresc la octetul 0.

3. Rezultate și probleme întâlnite

Rulând scripturile pe fișierele din probe/, am obținut toate rezultatele cerute. În xor.bin cheia este 0x80 (128), iar textul clar conține FLAG{xor_un_octet}. Cele cinci hash-uri md5 din hashuri.txt corespund parolelor password, dragon, qwerty, letmein și monkey. Fișierul challenge.bin este un base64 peste un XOR cu 0x80 și dă FLAG{lant_complet_despicat}, al cărui md5 (8c4eb02d9dcef7d0ff91d8e4c7fea91b) coincide cu cel din challenge_hint.txt. În mesaj.txt, textul trece prin base64, base64 și hex și ajunge la FLAG{straturi_de_encoding}. Din foto.jpg am aflat că poza a fost făcută cu un iPhone 13 (Apple) la 2026:09:14 10:30:00, la coordonatele 47.016667 N, 28.857500 E (Chișinău). Fișierul ascuns.png este de fapt un png cu un ZIP lipit la coadă: am extras arhiva gasit.zip de la octetul 782, iar ea conține FLAG{ascuns_prin_carving}. Din stego.png, creat din foto.jpg, am citit FLAG{ascuns_in_pixeli}. Imaginile arată la fel ca originalele, deci aceste date se descoperă doar prin analiză, nu cu ochiul liber.
Pe parcurs am găsit și câteva probleme în cod. Cea mai importantă este că desfă_multistrat se oprește când rezultatul intermediar este text imprimabil: pe mesaj.txt, după primul base64 obțin tot un șir base64, deci primesc NDY0Yzc3... în loc de FLAG{...}. Se rezolvă dacă continui decodarea cât timp rezultatul mai arată a strat cunoscut. În plus, testul test_sparge_xor pică, deoarece așteaptă o listă de candidați (cheie, text), iar sparge_xor returnează un singur tuplu; celelalte 5 teste trec. Filtrul „toți octeții imprimabili" nu dă nici o cheie unică (la challenge.bin aproape 40 de chei dau text imprimabil), așa că am confirmat cheia corectă cu md5-ul din hint. Tot așa, sparge_hash citește doar prima linie din hashuri.txt, deci sparge un singur hash din cinci; pe restul le-am spart separat, cu aceeași metodă. În sfârșit, căile sunt relative la probe/, deci scripturile merg doar din rădăcina proiectului, iar funcția getdata() din Pillow este depreciată.

CONCLUZII

În concluzie, am rezolvat toate țintele de laborator: în acest laborator am învățat că un fișier sau un șir arată mai mult decât pare la prima vedere. Am desfăcut un mesaj ascuns sub trei straturi de codare (base64, base64, hex), am spart un XOR cu cheie de un octet încercând toate cele 256 de chei și am aflat cinci parole din hash-uri MD5 cu un simplu dicționar. Apoi am citit tipul real al fișierelor după primii octeți, am scos din poză telefonul, data și coordonatele GPS (Chișinău), am extras o arhivă ZIP lipită la coada unui PNG și am ascuns un mesaj în ultimii biți ai pixelilor, fără ca poza să se schimbe vizibil. Cel mai greu a fost decodorul pe straturi: la început mă opream la primul text „citibil”, dar și un șir hex sau base64 e citibil, așa că lanțul nu ajungea la capăt, iar criteriul corect a fost să continui cât timp ce iese mai e o codare validă. Au mai ținut de dificultăți și lucrurile practice: să fiu în folderul potrivit când creez mediul virtual și instalez bibliotecile, să folosesc scriptul oficial de pregătire pentru ca probele să fie cele din fișă, și să deschid fișierele binare cu "rb". Aș putea da peste ele și în alte laboratoare, de exemplu cu chei XOR mai lungi sau parole care nu apar într-un dicționar. Ce rămâne după laborator este că diferența dintre aceste unelte și o infracțiune stă în permisiune, deci le folosesc doar pe fișiere proprii, iar înainte să public o poză îi șterg metadatele.

BIBLIOGRAFIE

https://docs.python.org/3/library/argparse.html
https://www.w3schools.com/Python/ref_module_base64.asp